

60. Un tableau, et une boucle « for »

Ranger beaucoup de choses.

1. Ce qu'il faut comprendre

`uint8_t cases[20];` réserve vingt octets en mémoire de travail — de quoi tenir un puits, une carte, un inventaire. Ils vivent hors de la page rapide, qui ne contient que les variables.

Un tableau `const` est différent : il est **gravé dans la cartouche**. Il ne prend pas un octet de mémoire, et ne peut plus changer — le compilateur refuse d'y écrire. C'est ce qu'on veut pour une table de formes, un niveau, une courbe.

Le `for` de C++ tient sur une ligne ce que le `while` demandait en quatre, et **la variable du compteur n'existe que dans la boucle**. Deux boucles voisines peuvent toutes deux nommer leur compteur `i` sans se marcher dessus : c'est la portée du langage, et le compilateur la tient vraiment.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <reste>` → [0.8.4. #include <reste> — a % b](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1066 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	49 o	← la console, à l'allumage — la mise en route de la console
main()	155 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
AttendreAcces	12 o	← EcrireTexte — attend de pouvoir écrire une case
AdresseCase	10 o	← poser() — trouve l'adresse d'une case de l'écran
EcrireTexte	10 o	← texte() — écrit un texte, lettre par lettre
Reste	10 o	← main() — le reste d'une division
CopierTuiles	18 o	← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage — vide les deux cartes de l'écran
Tuiles	704 o	← CopierTuiles — la police entière
"UN TABLEAU"	10 o	← texte() — un texte gravé
"ET UNE TABLE"	12 o	← texte() — un texte gravé
TABLE_0	10 o	← poser() — une table gravée

Lettres gravées : toutes (poser() reçoit un numéro calculé)

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EffacerCases, EcrireNombre, Diviser, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

poser() × 2 51 o sur place, dans main() + AdresseCase (10 o), TABLE_0 (10 o)

```
texte() × 2    22 o sur place, dans main() + EcrireTexte (10 o)
image() × 1    3 o sur place, dans main() + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <reste>    // a % b, sauf par 1, 2, 4, 8, 16... écrits en clair
#include <texte>    // écrit un texte à l'écran
#include <poser>    // pose une tuile sur une case du fond
```

2. UNE TABLE GRAVÉE DANS LA CARTOUCHE — TABLE

```
const uint8_t TABLE[] = { 1, 1, 2, 3, 5, 8, 3, 1, 4, 5 };
```

3. UN TABLEAU — CASES

```
uint8_t cases[20];
```

4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    texte(3, 3, "UN TABLEAU");
    texte(3, 11, "ET UNE TABLE");

    for (uint8_t i = 0; i < 20; i++) {
        cases[i] = i % 10;
    }

    for (uint8_t i = 0; i < 20; i++) {
        poser(i, 6, 27 + cases[i]);
    }

    for (uint8_t i = 0; i < 10; i++) {
        poser(i * 2, 14, 27 + TABLE[i]);
    }

    while (true) {
        image();
    }

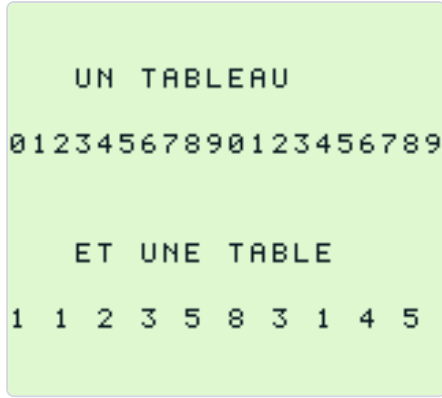
    return 0;
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Les chiffres 0 à 9 deux fois, puis la table gravée, un chiffre sur deux.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le tableau est rempli
- ✓ et il recommence après dix
- ✓ la table gravée se lit
- ✓ le titre est là

Ensuite : **60.1. La fonction `panneau()` — montrer le panneau** — chapitre 9 : Ranger ce qu'on manipule. · [gameboy3](#) — écrire une cartouche Game Boy en C++